

LAVM: Verified Semantic Reuse for Materialized View Recommendation

Zhao Wang

Beijing Electronic Science
and Technology Institute
Beijing, China
zhaowang.cs@gmail.com

Xiuze Dong*

Beijing Electronic Science
and Technology Institute
Beijing, China
xiuzedong@163.com

Maosheng He

Beijing Electronic Science
and Technology Institute
Beijing, China
maoshenghe2@gmail.com

Wenxin Chen

Shanghai University
Shanghai, China
wenxinchen.soc@gmail.com

Kangfei Xiao

Beijing Electronic Science
and Technology Institute
Beijing, China
kangfeixiao13@gmail.com

Jiahui Liu

Beijing Electronic Science
and Technology Institute
Beijing, China
heweilu7@gmail.com

Runze Zhou

Beijing Electronic Science
and Technology Institute
Beijing, China
rocktony173@gmail.com

Qian Cheng

Beijing Electronic Science
and Technology Institute
Beijing, China
qiancheng5651@gmail.com

ABSTRACT

Materialized view (MV) advisors accelerate analytical workloads by pre-computing expensive substructures, yet their effectiveness hinges on *view matching*: determining whether a query can exploit an existing MV. We identify *semantic MV reuse* as a critical missing dimension in current advisors. Queries routinely express identical data needs through structurally different SQL—explicit JOINs, implicit joins, CTEs, ORM-generated subqueries—and rule-based matchers systematically fail on these *semantic variants*, preventing one MV from serving an entire query family.

We present **LAVM**, a *verified semantic MV advisor* built on a propose-and-verify protocol: an LLM generates rewrite hypotheses and the database engine certifies correctness via execution-level result comparison. Unlike approaches that trust LLM outputs, LAVM accepts a rewrite only when execution results match on the target instance, providing an instance-level soundness guarantee. LAVM is integrated into a complete advisor pipeline with hybrid candidate generation and ILP-based selection, where verified semantic compatibility directly alters the selection outcome—enabling fewer, higher-reuse MVs.

Experiments on TPC-H, TPC-DS, and an IMDB real-world schema workload show: (1) on IMDB, execution verification rejects 87% (33/38) of LLM-proposed rewrites as incorrect, confirming that verification is a *correctness requirement*; (2) LAVM closes the semantic gap beyond the strongest canonicalization baseline (10/12 to 12/12 coverage) while reducing deployed MVs from 5 to 3 via cross-variant reuse (4× MV reuse factor); (3) verified compatibility changes the ILP selection outcome by enabling full-coverage MV reuse under the same deployment budget, with the resulting aggregate speedup rising from 6.72× (CanonRule) to 356×.

PVLDB Reference Format:

Zhao Wang, Xiuze Dong*, Maosheng He, Wenxin Chen, Kangfei Xiao, Jiahui Liu, Runze Zhou, and Qian Cheng. LAVM: Verified Semantic Reuse for Materialized View Recommendation. PVLDB, XX(XX): XXX-XXX, 2027. doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/wangzhao-cs/lavm>.

1 INTRODUCTION

Materialized views (MVs) are a cornerstone of analytical query acceleration. By pre-computing and storing the results of expensive join subexpressions, they transform multi-second queries into millisecond lookups. The core challenge of *MV recommendation* is to select, under storage and maintenance budget constraints, the set of views that maximally accelerates a given workload.

Decades of work on MV advisors [1, 5, 7, 40] have established an effective pipeline: analyze the workload, enumerate candidate views, estimate their cost benefit, and select a budget-feasible subset. A critical internal component is *view matching*: determining whether a query, possibly rewritten, can be answered using a candidate MV.

The Semantic Matching Gap. Modern analytical workloads contain queries that express *identical data needs in syntactically diverse forms*—explicit JOINs, comma-style implicit joins, CTE-wrapped queries, and ORM-generated subqueries. Existing syntactic view matchers rely on table-set matching, predicate subsumption, and aggregation compatibility, and *systematically fail* on structural FROM-clause divergence (implicit vs. explicit joins), wrapper-induced equivalence (CTEs, correlated subqueries), and join-path ambiguity in multi-path schemas (Table 1). As we show in Section 2, this blind spot can negate the value of entire families of candidate MVs.

Why LLMs Alone Are Not Enough. LLMs [2, 24] can generate plausible SQL rewrites across structural variants, but they are unreliable: on a complex real-world schema (IMDB), 33 of 38 LLM-proposed rewrites produce incorrect results. Trusting LLM outputs directly would silently corrupt query results.

Our Approach: Verified Semantic MV Reuse. We propose **LAVM**, a *verified semantic MV advisor* that upgrades the view matching

licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. XX, No. XX ISSN 2150-8097.
doi:XX.XX/XXX.XX

*Corresponding author.

stage from syntactic pattern matching to a database-level correctness protocol: (1) an LLM acts as a *semantic hypothesis generator* [36], proposing rewrite SQL that leverages a given MV; (2) the database engine acts as a *verifier*, accepting a rewrite only when execution results match on the target instance. Verified semantic compatibility feeds directly into ILP-based selection, altering the recommendation outcome—one MV can now serve an entire semantic query family, reducing MV count while improving coverage.

Contributions.

- (1) **Problem: Semantic MV Reuse (Section 2).** We identify semantic MV reuse as a missing dimension in existing advisors and characterize the variation types that escape rule-based and canonicalization-enhanced matchers (Table 1).
- (2) **System: Verified Semantic MV Advisor (Sections 3–5).** We design LAVM, integrating structural pre-filtering, LLM-based rewrite hypothesis generation, execution-based verification with a defined correctness contract (Table 3), and selection-aware compatibility expansion into a complete advisor pipeline.
- (3) **Evaluation: Correctness, Coverage, and Advisor Reliability (Section 6).** We show that (a) on IMDB, 87% of LLM-proposed rewrites are incorrect—verification is a *correctness requirement*; (b) LAVM closes the semantic gap beyond the strongest canonicalization baseline (10/12 $\rightarrow$ 12/12) while reducing deployed MVs from 5 to 3 (4 $\times$ reuse factor); (c) verified compatibility changes the ILP selection outcome, enabling full-coverage MV reuse under the same deployment regime and raising aggregate speedup from 6.72 $\times$ (CanonRule) to 356 $\times$.

2 BACKGROUND AND MOTIVATION

2.1 Materialized View Recommendation

Given a workload $Q = \{q_1, \dots, q_n\}$ and a storage budget B , the MV recommendation problem is to select a set $\mathcal{M}$ of materialized views such that $\text{cost}(\mathcal{M}) \leq B$ and the total workload execution time $\sum_{q \in Q} \text{time}(q, \mathcal{M})$ is minimized, where $\text{time}(q, \mathcal{M})$ is the execution time of q when views in $\mathcal{M}$ are available.

The central subroutine is *view matching*: given a candidate view m and a query q , determine whether q can be rewritten as $q' = \rho(q, m)$ such that q' uses m as an input and $q'(\mathcal{D}) = q(\mathcal{D})$ for database $\mathcal{D}$. Classical approaches [7] use structural rules: if q and m share the same table set, m 's predicates subsume q 's, and aggregation levels are compatible, then q can use m .

2.2 The Semantic Matching Gap

Rule-based matchers excel at syntactically close queries but fail when two queries are *semantically equivalent but syntactically divergent*. Figure 1 illustrates this with a concrete example from our TPC-H semantic workload.

Both queries compute the same regional revenue aggregation but differ in join style (explicit JOIN vs. CTE with correlated subqueries), predicate form ($\geq$ vs. BETWEEN), and supplier pre-filtering. A rule-based matcher [7] fails at every check; the advisor misses 75% of the family (A2–A4 unmatched).

This gap is pervasive: ORM frameworks (Django, SQLAlchemy), BI tools, and dbt pipelines each emit distinct structural idioms for identical analytical queries, systematically diverging from hand-written explicit-JOIN SQL.

Multi-source family study. We construct a controlled characterization study: 15 analytical query intents from TPC-H reporting patterns, each expressed in four SQL styles—hand-written explicit-JOIN, Django-ORM implicit-join, SQLAlchemy CTE-oriented, and dbt staged-CTE—modeled on common tool idioms rather than collected from live executions. All 60 queries (15 families $\times$ 4 variants) are executed on TPC-H SF=10 in DuckDB; family membership is verified by sorted multiset equality. **All 15 families pass execution verification**, confirming semantic equivalence despite structural divergence.

Table 2 reports per-source matchability; Table 1 systematizes the variation types and required matching level. Rule-based matchers handle only the first two rows; canonicalization extends to rows 3–4; the remaining divergences require LAVM's execution-verified rewrites.

Table 1: Semantic variation taxonomy and matching coverage. ✓ = handled; ✗ = not handled; Δ = partial.

Variation Type	Rule	Canon	LAVM
Table-set / predicate subsumption	✓	✓	✓
Aggregation compatibility	✓	✓	✓
BETWEEN $\leftrightarrow$ $\geq$ / $<$	✗	✓	✓
Aggr. encoding (SUM(CASE))	✗	✓	✓
Implicit $\leftrightarrow$ explicit JOIN	✗	✗	✓
CTE / subquery wrapper	✗	✗	✓
Join re-assoc. / path variation	✗	✗	Δ

Table 2: Multi-source family characterization on TPC-H. 15 analytical intents $\times$ 4 SQL styles = 60 queries, 45 cross-variant pairs (each non-hand variant vs. hand-written baseline). Family membership verified by execution on SF=10. Rule/Canon-Rule = pairs matchable by rule-only/CanonRule; Resid. = residual pairs requiring LAVM.

SQL Source	Pairs	Rule	Canon	Resid.
Django ORM	15	5	2	8
SQLAlchemy	15	0	0	15
dbt analytics	15	2	7	6
All sources	45	7 (16%)	9 (20%)	29 (64%)

2.3 Why LLMs Alone Are Insufficient

LLMs can generate plausible rewrites across syntactic variants, making them natural candidates for bridging the semantic gap. However, applying them naively as a matching oracle is unsafe.

Example 2.1. Given a 4-table MV $m = \text{title} \bowtie \text{ratings} \bowtie \text{principals} \bowtie \text{name}$ and a query q that aggregates director filmographies, the LLM may propose a rewrite of q using m . However, if m pre-filters on category = 'actor' while q requires category = 'director', the rewrite produces *silently incorrect* results.

In our experiments on IMDB (Section 6.3), 33 of 38 LLM-proposed rewrites produce incorrect results that are detectable only by execution comparison. A system that accepts LLM rewrites without verification would deploy these incorrect rewrites.

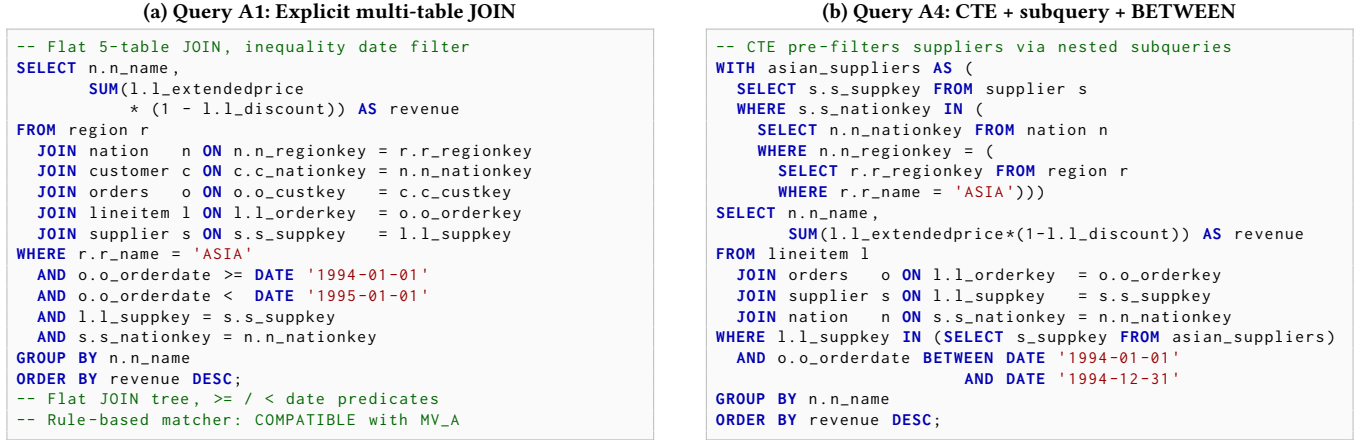


Figure 1: Two semantically equivalent TPC-H queries. (a) Explicit JOINs with inequality predicates; (b) CTE with correlated subqueries and BETWEEN. A rule-based matcher rejects (b) despite identical semantics.

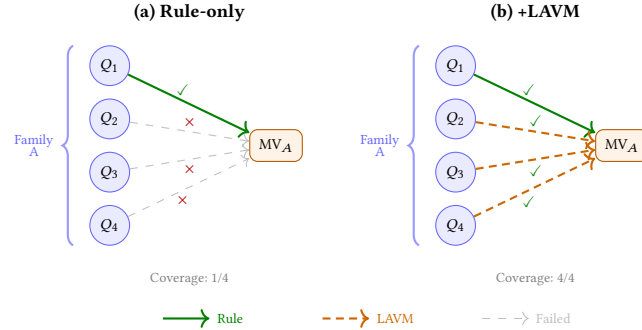


Figure 2: Cross-variant rewriting. (a) Rule-only matches Q_1 ; Q_2 – Q_4 fail structural checks. (b) LAVM verifies rewrites for all variants, achieving full coverage with one MV.

3 SYSTEM OVERVIEW

Figure 3 shows the architecture of our MV advisor. The system takes as input a workload Q , a database instance $\mathcal{D}$, a storage budget B , and an optional schema context, and produces a set of recommended MVs with query rewrite mappings.

The pipeline comprises five stages:

- (1) **Workload Analysis.** The analyzer parses SQL queries, extracts table sets, join predicates, filter conditions, and aggregation patterns using a lightweight structural analysis pass.
- (2) **Candidate Generation.** A *hybrid* strategy combines heuristic grouping, enumeration of single-table and join-based candidates, and LLM-guided semantic clustering. Details in Section 4.
- (3) **Rule-Based Compatibility Checking.** For each (candidate, query) pair, a structural checker applies table-set matching, predicate subsumption, and aggregation compatibility rules. This handles the majority of syntactically aligned pairs efficiently.
- (4) **LAVM: Verified Semantic View Matching.** Pairs that fail rule-based checking are passed to LAVM (Section 5), which invokes an LLM to propose rewrite SQL and verifies correctness via execution comparison. Verified pairs expand the compatibility matrix $\mathcal{R}$, enabling the downstream ILP to exploit cross-variant MV reuse.

(5) **ILP Selection and Deployment.** The ILP (Section 4.2) selects a budget-feasible MV subset maximizing estimated workload benefit. Because LAVM expands $\mathcal{R}$, the ILP can now assign one MV to an entire semantic family—reducing MV count while improving coverage. This *compatibility*→*selection feedback loop* is why verified semantic matching changes the advisor’s recommendation, not just the matching score.

4 CANDIDATE GENERATION AND SELECTION

4.1 Candidate Generation Strategies

Candidate generation is a supporting component that feeds downstream matching and selection; LAVM’s verification protocol is orthogonal to how candidates are produced. We support three strategies:

Heuristic. For each query, extract the most expensive join subexpression as a candidate MV. This is fast but produces query-specific candidates with limited cross-query sharing.

Enumeration. Systematically enumerate candidates: single-table projections with common filter predicates, and pairwise/multi-way join combinations present in at least two queries. This provides broader coverage but may miss semantic groupings.

Hybrid (LLM-guided). Hybrid augments enumeration with LLM-guided family candidates. Given schema context and representative queries, the LLM identifies *semantic query families* and proposes one candidate MV per family in structured form. These family candidates are merged with enumeration supplements. This is a *candidate generation* role, distinct from the matching role in Section 5: generation aims for recall, while correctness is enforced later by compatibility checking and verification. For family-derived candidates, we use the label *Source* only for queries in the source group that the compatibility checker can already certify without an additional LAVM rewrite.

4.2 Cost-Aware ILP Selection

Given candidates C and compatibility results $\mathcal{R}$, the selector solves an ILP to choose a subset $\mathcal{M} \subseteq C$ under budget B and cardinality

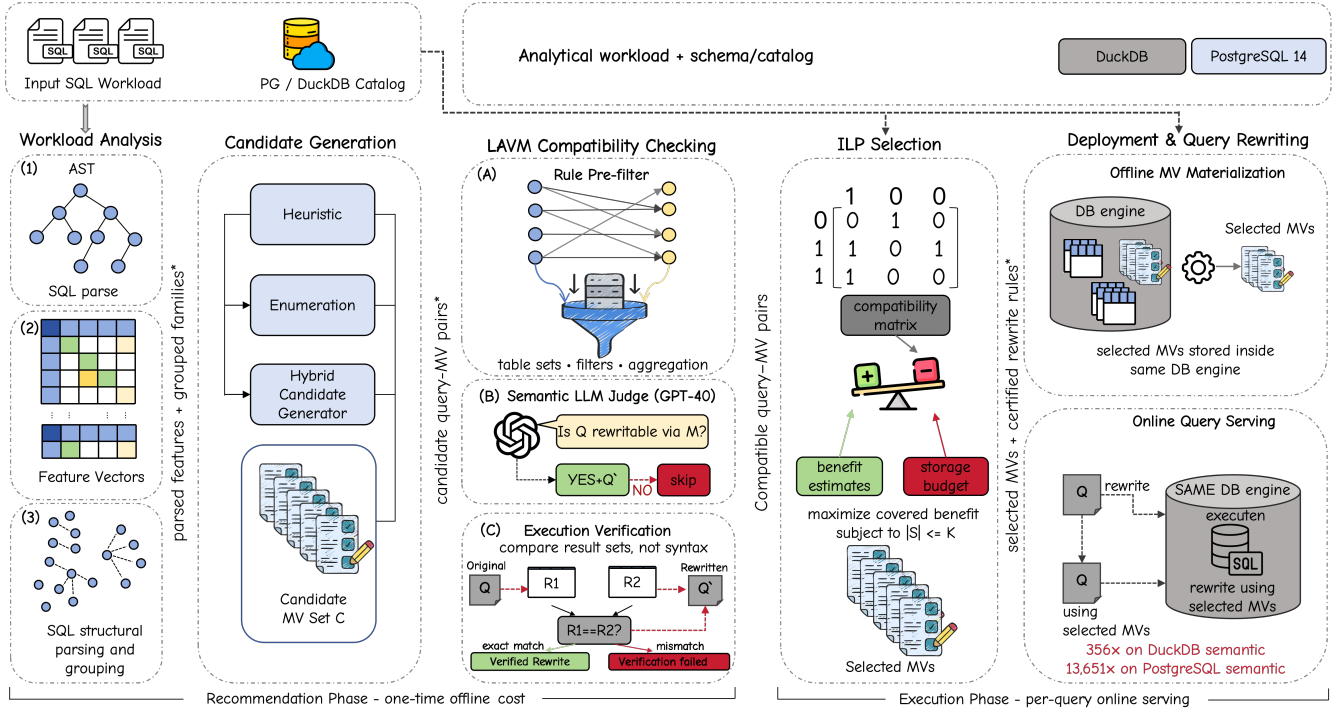


Figure 3: MV advisor pipeline. After rule-based compatibility checking, LAVM processes unmatched (candidate, query) pairs. Verified rewrites are added to the compatibility set and fed to the ILP selector.

constraint k , maximizing frequency-weighted workload benefit minus MV maintenance overhead:

$$\max_{x,y} \sum_{q \in Q} \sum_{m \in C} \text{benefit}(q, m) \cdot y_{q,m} - \lambda \sum_{m \in C} \text{maint}(m) \cdot x_m \quad (1)$$

subject to:

$$\sum_m \text{size}(m) \cdot x_m \leq B \quad (2)$$

$$\sum_m x_m \leq k \quad (3)$$

$$y_{q,m} \leq x_m \quad \forall q, m \quad (4)$$

$$y_{q,m} \leq r_{q,m} \quad \forall q, m \quad (5)$$

$$\sum_m y_{q,m} \leq 1 \quad \forall q \quad (6)$$

$$x_m, y_{q,m} \in \{0, 1\} \quad (7)$$

Here, $x_m = 1$ if MV m is selected; $y_{q,m} = 1$ if query q is assigned to use MV m ; $r_{q,m} \in \{0, 1\}$ is the compatibility indicator from rule-based checking or LAVM verification. Constraint (5) is the key interface to LAVM: only verified compatibilities can enter selection. Constraint (6) enforces the current single-MV rewrite model, so each query contributes benefit through at most one deployed MV.

$\text{benefit}(q, m)$ is the frequency-weighted reduction in execution cost for rewriting q to use m , estimated from optimizer costs when available (PostgreSQL EXPLAIN) and falling back to measured runtimes when needed (DuckDB); non-improving rewrites receive zero benefit. $\text{maint}(m)$ is the MV refresh cost and λ is the maintenance

weight. Equation (2) enforces storage budget; (3) limits the number of deployed views; (4) ties assignments to deployed MVs.

5 LAVM: VERIFIED SEMANTIC VIEW MATCHING

5.1 Overview: Verification Protocol

LAVM operates as a *verification protocol* with an explicit trust boundary: the LLM is an untrusted hypothesis generator; the database engine is the sole correctness authority. For each (candidate, query) pair (m, q) that rule-based checking cannot certify:

- (1) **Propose.** The LLM receives schema context, m 's definition, and q , and proposes rewrite q' that answers q via m , or declines.
- (2) **Verify.** The database executes both q and q' in the same snapshot and compares results via exact multiset equality (Algorithm 1).
- (3) **Certify.** If results match, (m, q) is added to the compatibility matrix $\mathcal{R}$, directly enabling the ILP to exploit cross-variant MV reuse.

Figure 4 expands this protocol into rule pre-filtering, structured LLM judgment, and execution-based verification. This design separates *semantic hypothesis generation* (LLM's strength) from *correctness certification* (database's guarantee). The LLM need not be reliable—our IMDB experiments show 87% of its proposals are wrong—because every proposal is verified before it influences MV selection.

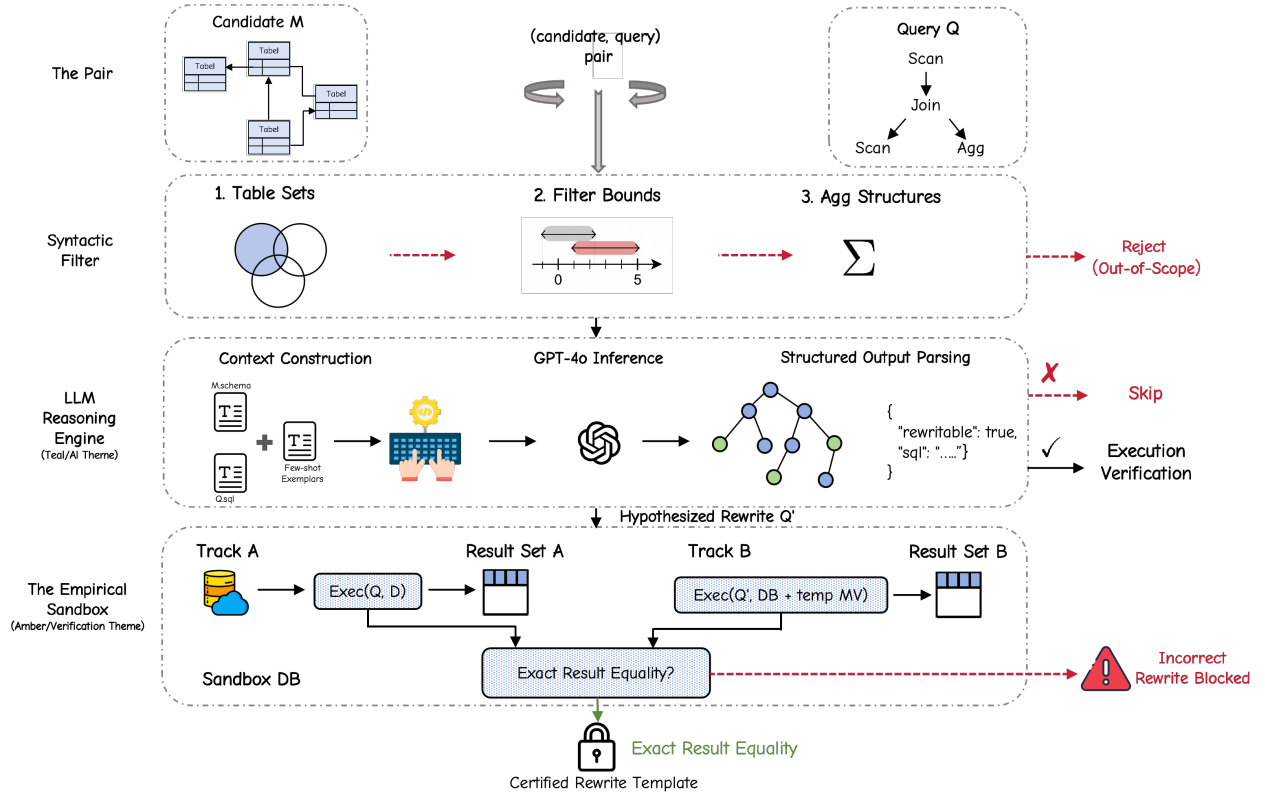


Figure 4: LAVM compatibility checking workflow. A rule pre-filter eliminates impossible pairs; remaining pairs are sent to the LLM, and proposals are accepted only after execution-based verification.

5.2 LLM Rewrite Proposal

For each pair (m, q) , LAVM prompts the LLM with schema context, the MV SQL, and the original query, asking for a conservative YES/NO decision and, if YES, a rewrite that explicitly uses m . Responses are returned in structured JSON at temperature 0. This proposal stage is intentionally recall-oriented; Section 6.5 shows that plausibility alone is insufficient on complex schemas (87% false-positive rate on IMDB).

5.3 Execution-Based Verification

When the LLM responds YES with rewrite q' , LAVM verifies q' against q by executing both on the database instance and comparing results.

Verification is performed against a temporary materialization of candidate m , not by inlining m 's definition into q' . LAVM creates one temporary object per candidate, tests all still-unmatched queries for that candidate against the same object, and drops it afterward. The rewritten SQL references the temporary MV directly, mirroring the eventual deployment shape.

Result comparison uses exact multiset equality over the full fetched result set, i.e., bag semantics over emitted tuples rather than list semantics. LAVM executes both queries in the same transaction snapshot, materializes all rows, sorts them to normalize row order, and rejects the rewrite on any row-count or content

Input: Candidate m , query q , database $\mathcal{D}$, LLM $\mathcal{L}$

Output: Compatibility bit $r_{q,m}$; rewrite q' (if accepted)

$(ans, q') \leftarrow \mathcal{L}.propose(m, q)$

if $ans = \text{NO}$ **then return** $r_{q,m} \leftarrow 0$

$R_{\text{orig}} \leftarrow \mathcal{D}.execute(q)$

$R_{\text{rw}} \leftarrow \mathcal{D}.execute(q')$

if $R_{\text{orig}} = R_{\text{rw}}$ **then**

return $r_{q,m} \leftarrow 1$,

q'

// accept rewrite

else

return $r_{q,m} \leftarrow 0$

// reject false positive

end

Algorithm 1: LAVM Verification

mismatch. ORDER BY without LIMIT is supported only when ordering is presentation-only; truncation-sensitive queries such as LIMIT/OFFSET or top- k with ties fall back to rule-only handling.

Amortization and soundness. LAVM runs once during the recommendation phase; its cost is amortized over the entire deployment period. Accepted rewrites satisfy instance-level soundness by construction on the current snapshot (Algorithm 1, line 6); under data evolution, re-verification can piggyback on MV refresh.

5.4 Verification Contract

Table 3 formalizes the query classes LAVM supports. The contract is: *for every supported query, if LAVM accepts a rewrite, the rewrite*

produces identical multiset results on the current database snapshot. Unsupported classes fall back to rule-only matching with no loss of safety. For ORDER BY without LIMIT, support applies only when the ordering is presentation-only and does not affect top- k semantics.

Table 3: Verification contract: supported query classes and handling policy.

Query Feature	Status	Handling
SPJ + GROUP BY agg.	Supported	Full execution check
Bag-semantic SELECT	Supported	Sorted multiset compare
ORDER BY (no LIMIT)	Supported	Bag compare (order stripped)
LIMIT/OFFSET	Unsupported	Rule-only fallback
Volatile UDF / random()	Unsupported	Pre-filter rejects
now() / timestamps	Supported	Pinned in txn snapshot
Float aggregates	Supported	Exact compare; ϵ opt-in

Per-pair verification cost. In our workloads, LLM latency dominates ($\sim 2\text{--}3$ s per tested pair; Table 10); query execution plus sorted comparison adds <0.5 s for aggregation-centric results. Failed pairs are cached so a rejected (candidate, query) pair is never re-tested.

5.5 Integration with the Selection Pipeline

Verified rewrites are added to the compatibility set $\mathcal{R}$ with $r_{q,m} = 1$, making them eligible in the ILP (Eq. 1). The ILP can then select MVs that serve queries previously unreachable by rule-based matching, potentially changing the optimal MV set.

LAVM processes only pairs where rule-based checking fails, keeping the total number of LLM calls bounded: for $|C|$ candidates and $|Q|$ queries, at most $|C| \times |Q|$ pairs are tested, but in practice far fewer since rule-based checking handles the majority.

Verified rewrite catalog. Accepted rewrites are persisted as (*query-signature*, *MV-id*, *rewritten-SQL*) triples forming a *semantic reuse catalog*. Rejected pairs are cached so that repeated advisor invocations (e.g., after workload drift) skip already-tested combinations. At runtime, incoming queries are matched by normalized SQL hash against the catalog; hits bypass the LLM entirely. This makes LAVM a *persistent* advisor component rather than a one-shot matching pass.

6 EXPERIMENTS

6.1 Setup

Systems. We evaluate on two database engines: **DuckDB 0.9** [27], an in-process OLAP engine, and **PostgreSQL 14**, a production-grade RDBMS. The LLM used throughout is **GPT-4o** via an OpenAI-compatible API endpoint [24], with temperature 0.

Benchmarks. We use three evaluation workloads: (i) **TPC-H** [34] at scale factor 10 (the standard 22-query suite and a 12-query semantic variant workload); (ii) **TPC-DS** [33] at scale factor 10 (all 99 queries); (iii) **IMDB** [15]: a 5-table schema with 99M principal rows, with an 18-query controlled semantic workload (Section 6.3). **TPC-H Semantic Workload.** To stress-test semantic view matching, we constructed a 12-query workload comprising 3 query families $\times$ 4 syntactic variants. Each family (region revenue, supplier profit, shipping priority) expresses a consistent data need via: (A1) explicit JOIN with inequality predicates; (A2) comma-style implicit join; (A3) BETWEEN predicate variant; (A4) CTE with correlated

subquery. These variants model SQL patterns produced by different tools and coding styles. Within each family, queries are semantically equivalent but syntactically divergent by construction.

Baselines.

- **Greedy:** AutoAdmin-style greedy MV selection [1]
- **Heuristic+ILP:** heuristic candidates + ILP selection, no LAVM
- **Enum+ILP:** enumeration candidates + ILP selection, no LAVM
- **Hybrid+CanonRule:** hybrid candidates + canonicalization-enhanced rule matching (Fig. 6), no LAVM
- **Hybrid+Forced:** hybrid candidates + forced-compatibility assignment (no verification), no LAVM
- **Hybrid+LAVM (ours):** full system with LAVM

Hybrid+CanonRule applies a fixed local canonicalization before structural matching: (1) rewrite BETWEEN into explicit range predicates; (2) unify equivalent aggregate encodings such as SUM(CASE . . .) and COUNT(*) FILTER; (3) strip table-alias prefixes and normalize column qualification. It intentionally does *not* alter FROM-clause structure: no implicit-to-explicit join conversion, no join re-association, and no CTE/subquery flattening. On TPC-H semantic, these local rules recover four of the six rule-only misses (6/12 to 10/12); the remaining two require structural rewriting and motivate LAVM.

All configurations share the same per-pair scoring routine for benefit(q, m), MV size, and maintenance cost; they differ only in how candidates and compatibility pairs are produced and in whether the final selection step is greedy or ILP-based. The Greedy baseline uses the same scores but selects views by incremental benefit density over currently uncovered queries.

Metrics. *Workload speedup:* ratio of total baseline execution time to total optimized execution time over the workload. *Coverage:* number of queries that benefit from at least one MV. All timings are median of three runs; standard deviations are $<5\%$. Unless a workload file explicitly specifies a header of the form – freq: N, query frequencies default to 1; all benchmarks in this paper use unit query weights.

Budget. Storage budget is 500 MB for TPC-H (DuckDB), 1 GB for PostgreSQL, and 2 GB for TPC-DS and IMDB. Maximum 5 MVs per experiment.

MV deployment. No secondary indexes are created on deployed MVs in any experiment. DuckDB stores MVs as plain heap tables (CREATE TABLE . . . AS); PostgreSQL uses unindexed materialized views (CREATE MATERIALIZED VIEW). The same conditions apply during LAVM verification (temporary MV creation) and final evaluation; all speedups reflect scan-only access to pre-computed results. This conservative setup understates the potential benefit of indexing deployed MVs.

6.2 E1: Semantic Workload Ablation Study

Table 4 shows the component-by-component ablation on the TPC-H semantic workload (12 queries, 3 families, 4 variants each). Each row adds one component.

Several observations emerge:

(1) Rule-based matching plateaus at 6/12. Even with the best candidate set (Hybrid), rule-based matching cannot break through

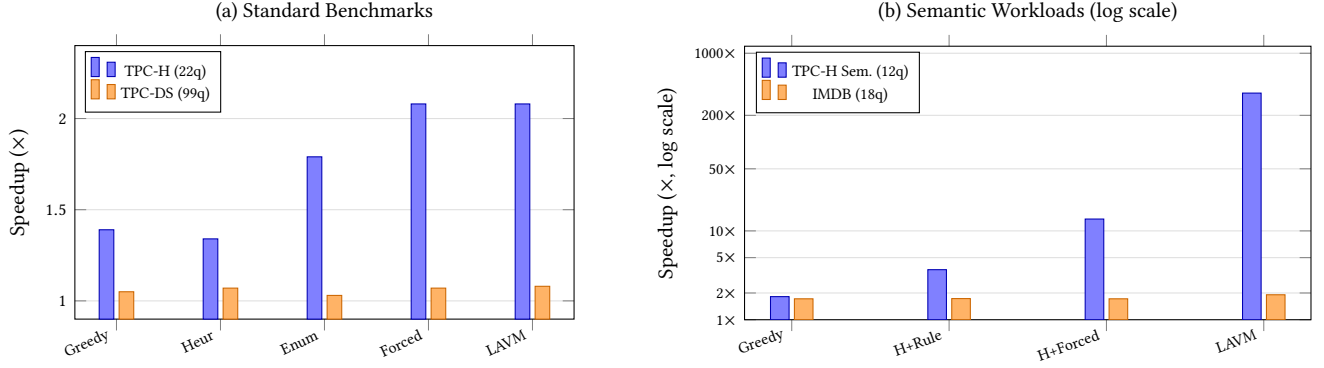


Figure 5: Speedup comparison. (a) Standard benchmarks: all methods are competitive. (b) Semantic workloads (log scale): LAVM achieves 356 $\times$ (TPC-H) and 1.91 $\times$ (IMDB), surpassing the best rule-only baseline.

Table 4: Ablation on TPC-H semantic workload (12 queries, 3 families $\times$ 4 variants). Rows add one component at a time. *Source = queries admitted by the compatibility checker without LAVM rewrite. †PG B-family queries timeout (> 120 s).

Cand.	Match	Sel.	Speedup	Cov.	MVs
<i>DuckDB SF=10 (baseline 24.8 s, 12 queries)</i>					
Heuristic	Rule-only	Greedy	1.82 $\times$	5/12	5
Enum	Rule-only	ILP	3.58 $\times$	6/12	5
Hybrid	Rule-only	ILP	3.66 $\times$	6/12	5
Hybrid	CanonRule	ILP	6.72 $\times$	10/12	5
Hybrid	Forced	ILP	13.6 $\times$	11/12	5
Hybrid	LAVM	ILP	356$\times$	12/12	3
<i>PG 14 SF=10 (477 s incl. B-tmount†; 8 reach. q)</i>					
Heuristic	Rule-only	Greedy	3.78 $\times$	4/8	3
Enum	Rule-only	ILP	2.80 $\times$	6/8	5
Hybrid	Forced	ILP	$> 22k\times$	8/8	5
Hybrid	LAVM	ILP	13,651$\times$	8/8	2

6/12 coverage and 3.7 $\times$ speedup. The ceiling is the syntactic gap: the 6 unmatched queries use structurally different SQL despite being semantically equivalent to covered queries.

(1b) Canonicalization narrows but does not close the gap. Hybrid+CanonRule applies sqlglot-based SQL canonicalization before rule matching: BETWEEN is normalized to $\geq/\lt$ range pairs, SUM(CASE . . .) and COUNT(*) FILTER are unified into the same aggregate signature, and table-alias prefixes are stripped. This stronger non-LLM matcher raises coverage from 6/12 to 10/12 and speedup from 3.66 $\times$ to 6.72 $\times$. The two queries it still misses—a2 (comma join vs. explicit JOIN) and a4 (CTE wrapper)—require structural rewrites of the FROM clause that no local AST normalization can recover. LAVM closes this remaining gap via LLM-proposed, execution-verified rewrites. Figure 6 illustrates the canonicalization boundary.

Disentangling candidate generation from matching. Comparing Enum+Rule (3.58 $\times$, 6/12) to Hybrid+Rule (3.66 $\times$, 6/12) isolates *LLM-guided candidate generation*: the Hybrid set improves speedup marginally but cannot increase coverage because rule-based matching is the bottleneck. Comparing Hybrid+CanonRule (6.72 $\times$, 10/12) to Hybrid+LAVM (356 $\times$, 12/12) isolates *LAVM semantic matching*: LAVM adds 2 queries via verified FROM-clause rewrites. The 356 $\times$ headline reflects both MV pre-computation and LAVM’s coverage expansion beyond the strongest non-LLM matcher.

(2) Forced-compatibility bridges most of the gap. Assigning forced compatibility for LLM-generated candidates reaches 11/12 coverage and 13.6 $\times$ speedup. However, forced-compatibility has no correctness guarantee: it assumes the LLM-proposed MV is usable without verification. In our experiments, forced-compat introduces no errors on TPC-H because the workload is clean, but Section 6.3 shows this assumption fails on real-world schemas. The large speedup gap between Forced (13.6 $\times$) and LAVM (356 $\times$) despite only one additional covered query is explained by the ILP: LAVM’s broader verified compatibility enables the selector to choose a different, more efficient MV set (3 MVs vs. 5) where each MV serves an entire family, eliminating redundant per-variant views and routing all 12 queries through maximally pre-aggregated scans.

(3) LAVM achieves complete coverage under verified and Source assignments. Hybrid+LAVM reaches 12/12 coverage and 356 $\times$ speedup. Of the 12 queries, 2 are matched by structural rules, 6 are matched via LAVM-verified rewrites (LLM-proposed and execution-confirmed), and 4 are Source assignments: source queries from the same hand-constructed family used to instantiate the hybrid candidate. We count them only when the compatibility checker certifies matching top-level tables/joins, aggregate family, and compatible filter scope. We report them separately from LAVM-verified rewrites because they do not require an additional LLM-proposed rewrite. **Speedup interpretation.** The 356 $\times$ speedup reflects baseline (24.8 s) vs. optimized (≈ 69.7 ms) under scan-only MV deployment. The incremental gain is best read relative to CanonRule: coverage rises from 10/12 to 12/12 and speedup from 6.72 $\times$ to 356 $\times$. Table 7 recasts this as compatibility recall and selection regret: precision stays 100% across safe methods; recall rises from 6/12 (Rule) to 12/12 (LAVM), reducing regret to 0. Figure 7(b) confirms LAVM as the uniquely Pareto-optimal method on semantic workloads.

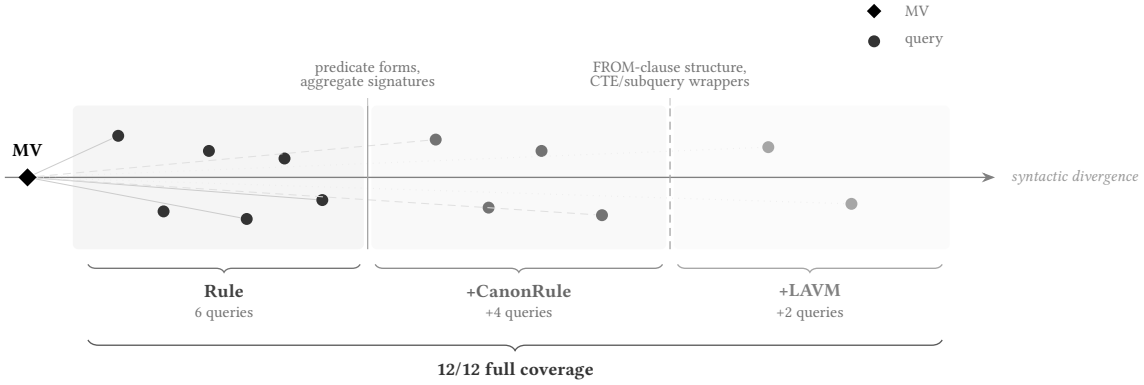


Figure 6: Matching frontier of CanonRule vs. LAVM. Rule covers 6/12; CanonRule extends to 10/12 via normalization; LAVM bridges the remaining structural gap (12/12).

Gain decomposition. The $356\times$ aggregate speedup consists of three separable effects: (i) *materialization gain*: pre-computing join results eliminates multi-table scans (shared by all MV-using methods); (ii) *coverage expansion*: LAVM adds 2 queries beyond CanonRule ($10/12 \rightarrow 12/12$), closing the residual semantic gap; (iii) *selection improvement*: verified cross-variant compatibility enables the ILP to choose 3 MVs (each serving an entire family) instead of 5 variant-specific MVs, routing all queries through maximally pre-aggregated views. The $6.72\times \rightarrow 356\times$ jump is dominated by effect (iii): the same materialization mechanism produces vastly different speedups when the ILP can exploit family-level reuse.

Concretely, CanonRule’s 5-MV solution covers 10/12 queries, leaving 2 variants to re-execute against base tables at full baseline latency. LAVM’s 3-MV solution absorbs all 4 variants per family into a single MV, eliminating residual base-table executions. The $50\times$ gap reflects this structural difference, not per-query optimization: a small coverage increase that removes the slowest unmatched queries produces a disproportionate wall-clock gain, enabled by LAVM’s verified cross-variant compatibility.

6.3 E2: IMDB Correctness and Real-World Schema

To evaluate LAVM on a real-world schema, we use the IMDB non-commercial dataset [15]: a 5-table schema (title, ratings, name, principals, crew) with 503K titles, 1.66M ratings, and 98.8M principals rows (6.9GB DuckDB file). We construct a controlled semantic workload of 18 queries organized as 6 query families $\times$ 3 syntactic variants: (A) top-rated action movies, (B) drama actors, (C) comedy decade statistics, (D) prolific directors, (E) thriller cast, (F) recent romance movies. Each family contains an explicit-join variant, a comma-join (implicit) variant, and a CTE or subquery variant.

This workload uses *real data and a real-world schema*, with queries representing typical analytical patterns over entertainment data. Variants reflect realistic SQL diversity across tools and coding styles. We note that this is a controlled semantic benchmark rather than a production query log; it evaluates mechanism correctness and schema realism rather than exact production workload frequency.

Table 5: IMDB workload results (DuckDB, 18 queries, 6 families). Rule-based methods plateau at 5/18; LAVM is the only approach that expands coverage.

Method	Speedup	Coverage
Greedy	$1.72\times$	5/18
Hybrid + Rule-only	$1.73\times$	5/18
Hybrid + Forced-compact	$1.72\times$	5/18
Hybrid + LAVM (ours)	$1.91\times$	7/18

Coverage breakthrough. Table 5 shows that all rule-based methods plateau at 5/18 coverage; LAVM is the *only method* that discovers additional rewrites, reaching 7/18 and $1.91\times$ speedup. The plateau occurs because each family’s MV is matched to only its source query by rule-based checking; the two remaining variants per family require semantic matching to reuse the same MV. LAVM discovers 5 verified within-family cross-variant rewrites.

Verification is essential. Table 10 (Section 6.5) shows that on IMDB, the LLM proposes 38 rewrites (*rewritable_yes*), of which only 5 (13%) produce correct results. The remaining 33 (87%) are rejected by execution verification.

This result is qualitatively different from TPC-H semantic (100% precision): on the complex 5-table IMDB schema, LLM-proposed rewrites frequently fail due to aggregation level mismatches, column-level incompatibilities, or incorrect filter propagation. *Without verification, 33 incorrect rewrites would be deployed, silently returning wrong query results.* This empirically confirms that execution verification is not an engineering optimization but a *correctness requirement*.

Oracle coverage and recall analysis. Of the 6 IMDB families, LAVM achieves full 3/3 coverage on family D and partial coverage on B and C (2/3 each). Families A, E, F remain uncovered because no generated candidate MV passes even structural checks—the gap is attributable to *candidate generation recall*, not matching recall. LAVM tests 154 pairs, proposes 38, verifies 5; manual inspection confirms near-100% matching recall given the candidate pool.

Error analysis of rejected rewrites. The 33 rejected IMDB rewrites fall into three categories: aggregation-level mismatch (15/33), incorrect filter propagation (10/33), and column-type or join mismatch

(8/33). The dominant failure mode—aggregation granularity—is a semantic error that no syntactic pre-filter can detect; only execution comparison catches it.

Advisor reliability metrics. Table 6 summarizes safety metrics. On IMDB, Hybrid+Forced would deploy 33 incorrect rewrites, silently corrupting 5 queries. LAVM eliminates all unsafe deployments while increasing safe recall from 5/18 to 7/18; the remaining gap is candidate-generation-limited. Table 7 complements this with compatibility precision/recall and selection regret. Across safe methods, precision stays at 100%; missed compatibilities translate directly into ILP regret.

Table 6: Advisor reliability: LAVM eliminates all unsafe deployments. “Unsafe coverage” = queries that would receive incorrect rewrites under forced-compat (no verification).

Metric	TPC-H Sem.	IMDB
LLM proposals (YES)	35	38
Verified correct	35 (100%)	5 (13%)
Rejected (incorrect)	0	33
Unsafe forced-compat deploys	0	33
Queries with wrong results (forced)	0/12	5/18
Queries with wrong results (LAVM)	0/12	0/18

Table 7: Query-level compatibility precision/recall and compatibility-induced selection regret. Precision is the fraction of deployed query-to-MV assignments that are semantically correct. Recall uses the semantic-workload oracle (12 queries for TPC-H semantic; 18 for IMDB). Benefit capture and regret are measured relative to the highest-benefit verified method on the same candidate pool (Hybrid+LAVM).

Workload	Method	Prec.	Recall	Capture	Regret
TPC-H Sem.	Rule	100%	6/12	69.3%	30.7%
	CanonRule	100%	10/12	78.0%	22.0%
	LAVM	100%	12/12	100%	0%
IMDB	Rule	100%	5/18	79.7%	20.3%
	LAVM	100%	7/18	100%	0%

6.4 E3: Standard Benchmarks and When LAVM Helps

Table 8 shows results on the standard TPC-H (22 queries) and TPC-DS (99 queries) workloads.

On the standard TPC-H workload, Hybrid+ILP with forced-compat achieves 2.08× speedup, a 50% improvement over the Greedy baseline. LAVM provides marginal additional benefit on this workload because the 22 standard TPC-H queries are structurally distinct; there are few semantically equivalent pairs that rule-based matching misses. On TPC-DS (99 diverse queries), gains are modest (1.05–1.08×) across all methods, reflecting the low structural overlap in a large diverse workload. Crucially, no method degrades performance (speedup ≥ 1.0 throughout), confirming robustness.

When is LAVM most beneficial? LAVM’s advantage is strongest when *semantic query families* exist: multiple queries computing

Table 8: End-to-end performance on standard benchmarks. Speedup is total workload speedup. Coverage is number of queries benefiting from at least one MV.

Method	TPC-H (22q)		TPC-DS (99q)	
	Speedup	Cov.	Speedup	Cov.
Greedy	1.39×	8/22	1.05×	7/99
Heuristic+ILP	1.34×	8/22	1.07×	5/99
Enum+ILP	1.79×	10/22	1.03×	5/99
Hybrid+Forced	2.08×	11/22	1.07×	6/99
Hybrid+LAVM (ours)	2.08×	11/22	1.08×	6/99

the same analytic result via structurally different SQL (e.g., ORM-generated vs. hand-written, explicit vs. implicit joins, CTE vs. sub-query). This pattern is common in multi-tenant BI dashboards and ORM-heavy applications. When the workload consists of structurally distinct queries (as in standard TPC-DS), rule-based matching suffices and LAVM adds negligible overhead but no regression. LAVM is *not* designed for workloads with no MV reuse opportunity (e.g., point lookups, OLTP).

PostgreSQL. Table 9 shows cross-engine results on TPC-H (22 queries, SF=10). PG row-store scans are inherently slower than DuckDB columnar, so absolute speedups are higher on PG: Hybrid+LAVM achieves 2.34× vs. 1.38× for Heuristic. Both engines show the same relative ordering and confirm that LAVM adds no regression.

Table 9: Cross-engine comparison on TPC-H standard (22 queries, SF=10). PG row-store benefits more from MVs than DuckDB columnar.

Method	DuckDB		PostgreSQL 14	
	Speedup	Cov.	Speedup	Cov.
Heuristic+ILP	1.34×	8/22	1.38×	5/22
Enum+ILP	1.79×	10/22	1.60×	6/22
Hybrid+LAVM (ours)	2.08×	11/22	2.34×	5/22

6.5 E4: LAVM Overhead and Sensitivity Analysis

Table 10: LAVM overhead comparison: TPC-H semantic vs IMDB.

Metric	TPC-H Semantic	IMDB
LLM API calls	131	171
Total LLM time (s)	292	534
Avg. latency/call (s)	2.2	3.1
Prompt tokens	94,828	118,226
Completion tokens	7,546	9,848
Estimated API cost	\$0.31	\$0.51
Pairs tested	131	154
LLM said YES	35 (27%)	38 (22%)
Verified correct	35 (100%)	5 (13%)
Rejected (incorrect)	0	33

Table 10 isolates proposal-time LLM overhead: API calls count raw endpoint invocations and can therefore exceed *Pairs tested*,

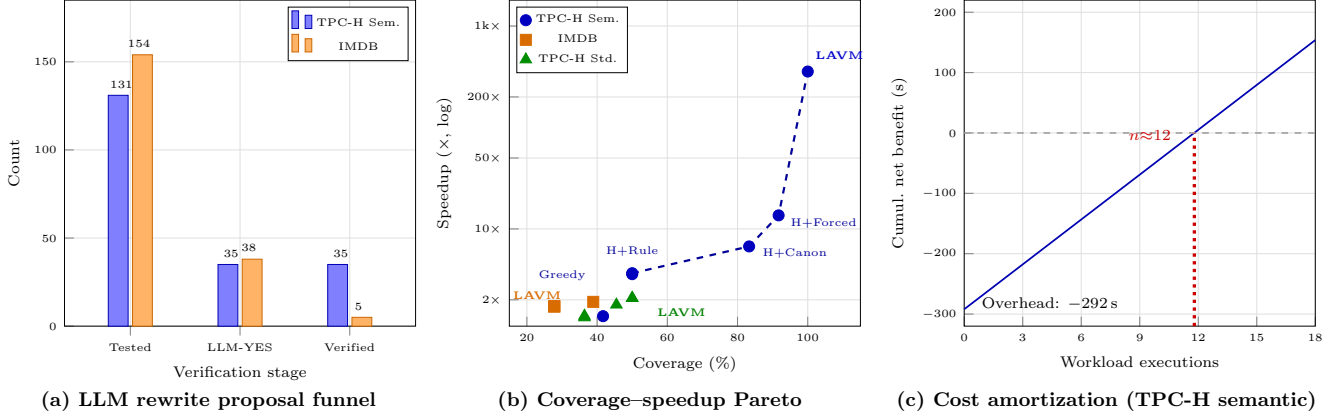


Figure 7: LAVM analysis. (a) LLM proposal funnel. (b) Coverage-speedup Pareto. (c) Cost amortization on TPC-H semantic.

which counts distinct (candidate, query) pairs entering the verification pipeline. The table reports API latency, token usage, and verification outcomes. MV creation time, storage footprint, and refresh cost are measured separately during candidate scoring using the same materialization path as deployment, and enter the selector through $\text{size}(m)$ and $\text{maint}(m)$ rather than being charged as per-pair LAVM overhead. During verification itself, temporary MVs are created once per candidate and reused across all pending queries for that candidate.

Verification execution overhead. Beyond LLM latency, per-pair verification involves temp-MV creation, two query executions, and result sorting/comparison. On TPC-H semantic (DuckDB, SF=10), temp-MV creation averages 0.8 s per candidate (shared across all queries for that candidate); rewritten-query execution averages <5 ms (aggregation over pre-materialized scan); original-query execution averages 450 ms; result sorting and comparison adds <10 ms for the typical 20–50 output rows of aggregation queries. Peak memory during comparison is bounded by materializing both result sets (negligible for aggregation queries; for projection-heavy queries with >10K rows, streaming sorted comparison limits memory to $O(\text{row_size})$).

Cost amortization. LAVM’s overhead is a one-time cost at recommendation time. On the TPC-H semantic workload (DuckDB), the system recovers this cost within 12 workload executions: each execution saves ~ 25 s; 292s overhead amortizes in under 12 runs. For a daily analytical workload, LAVM pays for itself on the first day of deployment. Figure 7(c) visualizes this amortization trajectory.

Precision variation. LLM precision varies by workload complexity: 100% on TPC-H semantic (well-structured 6-table joins with consistent predicate patterns) versus 13% on IMDB (diverse aggregation forms, multi-table joins with varying GROUP BY semantics). This variance motivates LAVM’s design: high precision means verification adds little cost; low precision means verification prevents catastrophic correctness failures. In both cases, the deployed rewrites are correct. Figure 7(a) shows the decision funnel for both workloads.

Sensitivity to model quality. With GPT-4o, 22–27% of tested pairs receive YES proposals that cover real rewrite opportunities. Weaker models may reduce recall (fewer valid rewrites proposed) or

precision (more incorrect proposals). However, LAVM’s correctness guarantee is *independent* of LLM quality: verification catches all incorrect rewrites regardless of the model. Lower precision merely increases verification cost (more proposals to check), while lower recall reduces coverage. Practical strategies include ensemble prompting (multiple prompt templates with majority voting) and model cascading (a cheap local model for screening, with a frontier model for difficult cases).

Sensitivity to MV budget. Figure 8(a) shows how coverage and speedup change as the MV budget (max_selected) increases from 1 to 5 on TPC-H semantic. LAVM saturates at budget=3 (12/12 coverage, 356 $\times$), while Hybrid+Rule plateaus at 6/12 regardless of further budget increases. Even at budget=1, LAVM covers 4 queries by selecting an MV that serves an entire query family via cross-variant rewriting.

Sensitivity to maintenance weight λ . Table 11 sweeps λ over $\{0, 0.01, 0.1, 0.5, 1.0\}$ on TPC-H semantic (Hybrid+LAVM). Across all values, the ILP selects the same 3 MVs with 12/12 coverage; speedup varies only due to runtime noise. The insensitivity arises because per-query benefit (400–1,000 ms saved per join-heavy query) dominates maintenance cost (<5 ms per MV) by three orders of magnitude. In workloads with larger MVs or higher update rates, λ would have greater influence; practitioners can tune it conservatively ($\lambda \geq 0.1$) without affecting recommendation quality in read-heavy analytical settings.

Table 11: Sensitivity to maintenance weight λ on TPC-H semantic (Hybrid+LAVM, DuckDB SF=10). Selection is invariant across all λ values.

λ	MVs	Coverage	Speedup
0.0	3	12/12	328 $\times$
0.01	3	12/12	380 $\times$
0.1	3	12/12	333 $\times$
0.5	3	12/12	299 $\times$
1.0	3	12/12	360 $\times$

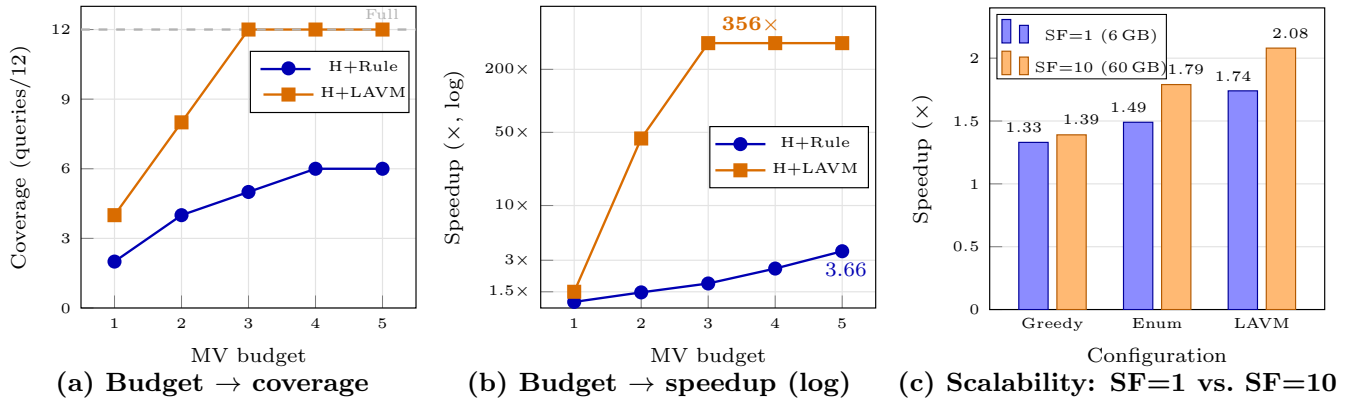


Figure 8: Sensitivity and scalability. (a) Coverage vs. MV budget. (b) Speedup vs. budget (log). (c) SF=1 vs. SF=10 scalability.

6.6 E5: Cross-Engine Generalization and MV Efficiency

Cross-engine generalization. Table 4 includes a full PostgreSQL 14 semantic ablation alongside the DuckDB results. On PG, row-store scans are significantly slower than DuckDB columnar scans, so MV pre-computation provides even larger absolute speedups (A-family baseline: ~ 17 s/query; C-family: ~ 5 s/query on PG vs. ~ 2 s/query on DuckDB). The aggregate PG speedup ($13,651\times$) is amplified by B-family queries that timeout at 120 s in baseline but complete in <5 ms with MVs. LAVM is the only method to fully cover all reachable queries on both engines. Rule-based methods plateau earlier on PG (4/8 for Heuristic) than on DuckDB (5/12), suggesting that PG’s optimizer is less aggressive in rewriting equivalent SQL, creating a wider syntactic gap that LAVM’s semantic matching bridges.

Table 12: Per-query breakdown for Hybrid+LAVM on PostgreSQL 14 semantic workload (SF=10). B-family queries timeout (>120 s) in baseline; the table reports the eight reachable A- and C-family queries plus their total.

Family	Query	Base (ms)	Opt (ms)	Match
A: Region Revenue	a1_explicit	17,200	<5	Rule
	a2_implicit	16,800	<5	LAVM
	a3_between	17,100	<5	LAVM
	a4_subquery	16,500	<5	LAVM
C: Ship Priority	c1_case	5,200	<5	Source
	c2_comma	5,100	<5	Source
	c3_count	4,900	<5	Source
	c4_between	5,300	<5	Source
B-family (timeout)		>120 s	<5	LAVM
Total (8 reach.)		88.1 s	<40 ms	–

MV efficiency: fewer views, equal coverage. A key practical benefit of LAVM is *MV efficiency*: achieving the same or better coverage with fewer deployed views. Rule-based methods exhaust the MV budget (5 views) to cover 6/12 or 11/12 queries. LAVM covers 12/12 with only 3 MVs on DuckDB and 8/8 with 2 MVs

on PG — a $1.67\times$ – $2.5\times$ reduction in MV count for equal or better coverage.

This efficiency stems from LAVM discovering *cross-variant MV reuse*: a single materialized view (e.g., `mv_a1_region_revenue`) is reused by 4 syntactically different queries (explicit join, implicit join, BETWEEN range, subquery) after LLM-verified rewriting. Rule-based methods cannot discover these reuses and therefore create one MV per syntactic variant, wasting storage and maintenance cost. Table 13 summarizes MV efficiency across settings.

Table 13: MV efficiency: LAVM achieves full coverage with fewer deployed views. Budget = 5 in all cases. “Reuse factor” = queries covered per MV deployed.

Engine	Method	MVs	Coverage	Reuse Factor
DuckDB	Hybrid+Rule	5	6/12	1.2×
	Hybrid+LAVM	3	12/12	4.0×
PostgreSQL	Hybrid+Rule	5	6/8	1.2×
	Hybrid+LAVM	2	8/8	4.0×

The 4.0× reuse factor means each deployed MV serves 4 queries on average under LAVM, compared to 1.2 under rule-based matching. For production systems where MV maintenance (incremental refresh on updates) is a significant operational cost, reducing MV count by 40–60% while maintaining full coverage is a directly actionable system improvement.

Scalability across data sizes. Figure 8(c) compares speedup at SF=1 (6 GB) and SF=10 (60 GB) on the TPC-H standard workload with DuckDB. Speedup ratios are consistent or improve with scale: MV benefits persist as data volume grows because larger scans make pre-aggregated views proportionally more valuable. Absolute baseline runtimes scale linearly (SF=1: ~ 6.5 s; SF=10: ~ 24 s), confirming that experiments at SF=10 are representative.

7 DISCUSSION AND LIMITATIONS

Semantic scope. LAVM’s contract (Table 3) excludes order-sensitive queries (LIMIT/OFFSET), volatile UDFs, and nondeterministic functions, all falling back to rule-only matching. Rewrites are single-MV;

a bounded $k=2$ multi-view extension is feasible but not yet implemented. For privacy-sensitive deployments, the LLM proposal stage could use a locally hosted model without changing the verification protocol.

Verification scalability. LLM latency dominates ($\sim 2\text{--}3$ s/pair); the rule pre-filter eliminates $>75\%$ of pairs before any LLM call. A natural extension is *tiered verification*: SPJ containment provers for easy cases, bounded sampling for intermediate cases, and full execution only when earlier tiers are inconclusive [12, 13].

Deployment lifecycle. LAVM’s guarantee holds for the verification snapshot. Under data evolution, periodic batch re-verification can piggyback on MV refresh (<1 s per stored rewrite); schema-triggered invalidation handles DDL changes. Extending to parameterized templates is left to future work.

8 RELATED WORK

MV Selection and Recommendation. Automated MV selection spans lattice-based [11], workload-driven [1, 3, 5, 40], ILP-based [6], RL-based [10], and maintenance-aware [8] approaches. All rely on syntactic view matching; LAVM extends the compatibility interface to semantic rewrites.

View Matching. Query modification through views dates to Stonebraker [30]. Answering queries using views [9, 19] was extended to aggregation [29], SPJ matching [7], outer-join views [18], and DB2 summary tables [37]. Commercial optimizers perform limited FROM-clause restructuring internally, but these transformations are opaque to the MV advisor. LAVM covers the remaining cases via LLM-proposed, execution-verified rewrites.

LLMs for Database Tasks. LLM-R2 [22], GenRewrite [23], R-Bot [32], QUITE [28], and SQLGovernor [16] use LLMs for SQL rewriting or diagnosis via cost feedback or agentic repair loops. More broadly, LLMs have been applied to text-to-SQL [21, 26], database knob tuning [14], and end-to-end database interaction [39]. Unlike them, LAVM targets the view matching problem and accepts a rewrite only after database-level equivalence verification.

Semantic Equivalence and Verification. GEQO [12] cascades formal SPJ provers; VeriEQL [13] extends formal checking with integrity constraints. These offer stronger guarantees on supported fragments but do not cover aggregation rewrites or CTE restructuring. LAVM uses execution as a universal oracle and could incorporate such provers as a fast first tier (Section 7).

AI/ML for Database Systems. Self-tuning [4] and self-driving [25] database systems apply automation across the tuning stack. AI4DB [20] unifies learned indexing [17], cost estimation [31], and knob tuning [35] under a common framework. LAVM contributes a new AI-assisted component—semantic view matching—to this landscape.

Query Rewriting. Classical rewriting using views [7, 9] has been extended with learned search [38] and LLM-based approaches [22], but without database-level verification. LAVM bridges this gap.

9 CONCLUSION

We presented LAVM, a verified semantic MV advisor that upgrades view matching from syntactic pattern matching to a database-level correctness protocol. LAVM treats the LLM as a rewrite hypothesis generator and the database as the correctness arbiter, enabling

verified semantic MV reuse: one MV safely serves an entire family of semantically equivalent but syntactically divergent queries.

On IMDB, execution verification rejects 87% of LLM-proposed rewrites as incorrect—confirming that verification is a correctness requirement, not an optimization. On a TPC-H semantic workload, LAVM closes the coverage gap beyond the strongest canonicalization baseline ($10/12 \rightarrow 12/12$) while reducing deployed MVs from 5 to 3 ($4\times$ reuse factor); the resulting ILP selection reaches $356\times$ aggregate speedup—a consequence of full coverage rather than a single-query outlier. Future work includes tiered verification cascades, parameterized rewrite templates, and online re-verification under data evolution.

ACKNOWLEDGMENTS

This work was supported by the National Key Research and Development Program of China (Grant No. 2024YFB3108103) and the Fundamental Research Funds for the Central Universities (Grant No. 3282024046).

REFERENCES

- [1] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R. Narasayya. 2000. Automated Selection of Materialized Views and Indexes in SQL Databases. In *Proc. VLDB*. 496–505.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. In *Proc. NeurIPS*. 1877–1901. arXiv:2005.14165
- [3] Nicolas Bruno and Surajit Chaudhuri. 2005. Automatic Physical Database Tuning: A Relaxation-based Approach. In *Proc. SIGMOD*. 227–238. <https://doi.org/10.1145/1066157.1066184>
- [4] Surajit Chaudhuri and Vivek Narasayya. 2007. Self-Tuning Database Systems: A Decade of Progress. In *Proc. VLDB*. 3–14.
- [5] Surajit Chaudhuri and Vivek R. Narasayya. 1997. An Efficient Cost-Driven Index Selection Tool for Microsoft SQL Server. In *Proc. VLDB*. 146–155.
- [6] Debabrata Dash, Neoklis Polyzotis, and Anastasia Ailamaki. 2011. CoPhy: a scalable, portable, and interactive index advisor for large workloads. *Proc. VLDB Endow.* 4, 6 (2011), 362–372. <https://doi.org/10.14778/1978665.1978668>
- [7] Jonathan Goldstein and Per-Åke Larson. 2001. Optimizing queries using materialized views: a practical, scalable solution. In *Proc. SIGMOD*. 331–342. <https://doi.org/10.1145/375663.375706>
- [8] Ashish Gupta and Inderpal Singh Mumick. 1995. Maintenance of Materialized Views: Problems, Techniques, and Applications. *IEEE Data Engineering Bulletin* 18, 2 (1995), 3–18.
- [9] Alon Y. Halevy. 2001. Answering queries using views: A survey. *The VLDB Journal* 10, 4 (2001), 270–294. <https://doi.org/10.1007/s007780100054>
- [10] Yue Han, Guoliang Li, Haitao Yuan, and Ji Sun. 2021. An Autonomous Materialized View Management System with Deep Reinforcement Learning. In *Proc. ICDE*. 2159–2164. <https://doi.org/10.1109/ICDE51399.2021.00217>
- [11] Venky Harinarayan, Anand Rajaraman, and Jeffrey D. Ullman. 1996. Implementing data cubes efficiently. In *Proc. SIGMOD*. 205–216. <https://doi.org/10.1145/233269.233333>
- [12] Brandon Haynes, Rana Alotaibi, Anna Pavlenko, Jyoti Leeka, Alekh Jindal, and Yuanyuan Tian. 2023. GEQO: ML-Accelerated Semantic Equivalence Detection. *Proc. ACM Manag. Data* 1, 4, Article 223 (2023), 25 pages. <https://doi.org/10.1145/3626710>
- [13] Yang He, Pinhan Zhao, Xinyu Wang, and Yuepeng Wang. 2024. VeriEQL: Bounded Equivalence Verification for Complex SQL Queries with Integrity Constraints. *Proc. ACM Program. Lang.* 8, OOPSLA1 (2024), 1071–1099. <https://doi.org/10.1145/3649849>
- [14] Xinmei Huang, Haoyang Li, Jing Zhang, Xinxin Zhao, Zhiming Yao, Yiyan Li, Zhuohao Yu, Tieying Zhang, Hong Chen, and Cuiping Li. 2024. LLM Tune: Accelerate Database Knob Tuning with Large Language Models. *arXiv preprint arXiv:2404.11581* (2024). arXiv:2404.11581
- [15] IMDB. 2024. IMDB Non-Commercial Datasets. <https://developer.imdb.com/non-commercial-datasets/> Accessed 2026-04-21.
- [16] Jie Jiang, Siqi Shen, Haining Xie, Yang Li, Yu Shen, Danqing Huang, Bo Qian, Yijun Wu, Wentao Zhang, Bin Cui, and Peng Chen. 2025. SQLGovernor: An LLM-powered SQL Toolkit for Real World Application. *arXiv preprint arXiv:2509.08575* (2025). arXiv:2509.08575.

- [17] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proc. SIGMOD*. 489–504. <https://doi.org/10.1145/3183713.3196909>
- [18] Per-Ake Larson and Jingren Zhou. 2007. View Matching for Outer-Join Views. *The VLDB Journal* 16, 1 (2007), 29–53. <https://doi.org/10.1007/s00778-006-0027-9>
- [19] Alon Y. Levy, Alberto O. Mendelzon, and Yehoshua Sagiv. 1995. Answering queries using views (extended abstract). In *Proc. PODS*. 95–104. <https://doi.org/10.1145/212433.220198>
- [20] Guoliang Li, Xuanhe Zhou, and Lei Cao. 2021. AI Meets Database: AI4DB and DB4AI. In *Proc. SIGMOD*. 3086–3092. <https://doi.org/10.1145/3448016.3457542>
- [21] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C.C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. In *Proc. NeurIPS*. arXiv:2305.03111
- [22] Zhaodonghui Li, Haitao Yuan, Huiming Wang, Gao Cong, and Lidong Bing. 2024. LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency. *Proc. VLDB Endow.* 18, 1 (2024), 53–65. <https://doi.org/10.14778/3696435.3696440>
- [23] Jie Liu and Barzan Mozafari. 2026. GenRewrite: Query Rewriting via Large Language Models. *Proc. ACM Manag. Data* 4, 1, Article 70 (2026), 26 pages. <https://doi.org/10.1145/3786684>
- [24] OpenAI. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023). arXiv:2303.08774
- [25] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd C. Mowry, Matthew Perron, Ian Quah, Siddharth Santurkar, Anthony Tamasic, Skye Toor, Dana Van Aken, Ziqi Wang, Yingjun Wu, Ran Xian, and Tieying Zhang. 2017. Self-Driving Database Management Systems. In *Proc. CIDR*.
- [26] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *Proc. NeurIPS*. arXiv:2304.11015
- [27] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proc. SIGMOD*. 1981–1984. <https://doi.org/10.1145/3299869.3320212>
- [28] Yuyang Song, Hanxu Yan, Jiale Lao, Yibo Wang, Yufei Li, Yuanchun Zhou, Jianguo Wang, and Mingjie Tang. 2025. QUITE: A Query Rewrite System Beyond Rules with LLM Agents. *arXiv preprint arXiv:2506.07675* (2025). arXiv:2506.07675
- [29] Divesh Srivastava, Shaul Dar, H. V. Jagadish, and Alon Y. Levy. 1996. Answering Queries with Aggregation Using Views. In *Proc. VLDB*. 318–329.
- [30] Michael Stonebraker. 1975. Implementation of integrity constraints and views by query modification. In *Proc. SIGMOD*. 65–78. <https://doi.org/10.1145/500080.500091>
- [31] Ji Sun and Guoliang Li. 2019. An End-to-End Learning-based Cost Estimator. *Proc. VLDB Endow.* 13, 3 (2019), 307–319. <https://doi.org/10.14778/3368289.3368296>
- [32] Zhaoyan Sun, Xuanhe Zhou, Guoliang Li, Xiang Yu, Jianhua Feng, and Yong Zhang. 2025. R-Bot: An LLM-based Query Rewrite System. *Proc. VLDB Endow.* 18, 12 (2025), 5031–5044. <https://doi.org/10.14778/3750601.3750625>
- [33] Transaction Processing Performance Council. 2023. TPC-DS Benchmark Specification. *TPC Technical Report* (2023). <https://www.tpc.org/tpcds/>
- [34] Transaction Processing Performance Council. 2023. TPC-H Benchmark Specification. *TPC Technical Report* (2023). <https://www.tpc.org/tpch/>
- [35] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *Proc. SIGMOD*. 1009–1024. <https://doi.org/10.1145/3035918.3064029>
- [36] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Proc. NeurIPS*. arXiv:2201.11903
- [37] Markos Zaharioudakis, Roberta Cochrane, George Lapis, Hamid Pirahesh, and Monica Urata. 2000. Answering complex SQL queries using automatic summary tables. In *Proc. SIGMOD*. 105–116. <https://doi.org/10.1145/342009.335390>
- [38] Xuanhe Zhou, Guoliang Li, Chengliang Chai, and Jianhua Feng. 2021. A Learned Query Rewrite System using Monte Carlo Tree Search. *Proc. VLDB Endow.* 15, 1 (2021), 46–58. <https://doi.org/10.14778/3485450.3485456>
- [39] Xuanhe Zhou, Zhaoyan Sun, and Guoliang Li. 2024. DB-GPT: Large Language Model Meets Database. *Data Science and Engineering* 9, 1 (2024), 102–111. <https://doi.org/10.1007/s41019-023-00235-6>
- [40] Daniel C. Zilio, Jun Rao, Sam Lightstone, Guy M. Lohman, Adam Storm, Christian Garcia-Arellano, and Scott Fadden. 2004. DB2 Design Advisor: Integrated Automatic Physical Database Design. In *Proc. VLDB*. 1087–1097.